

Deep Learning

Getting Started with Neural Networks: Classification and Regression



Prof. Anxiao (Andrew) Jiang

Learning Objectives:

1. Understand deep learning for binary classification.
2. Understand deep learning for multi-class classification.
3. Understand deep learning for regression.



Roadmap of this lecture:

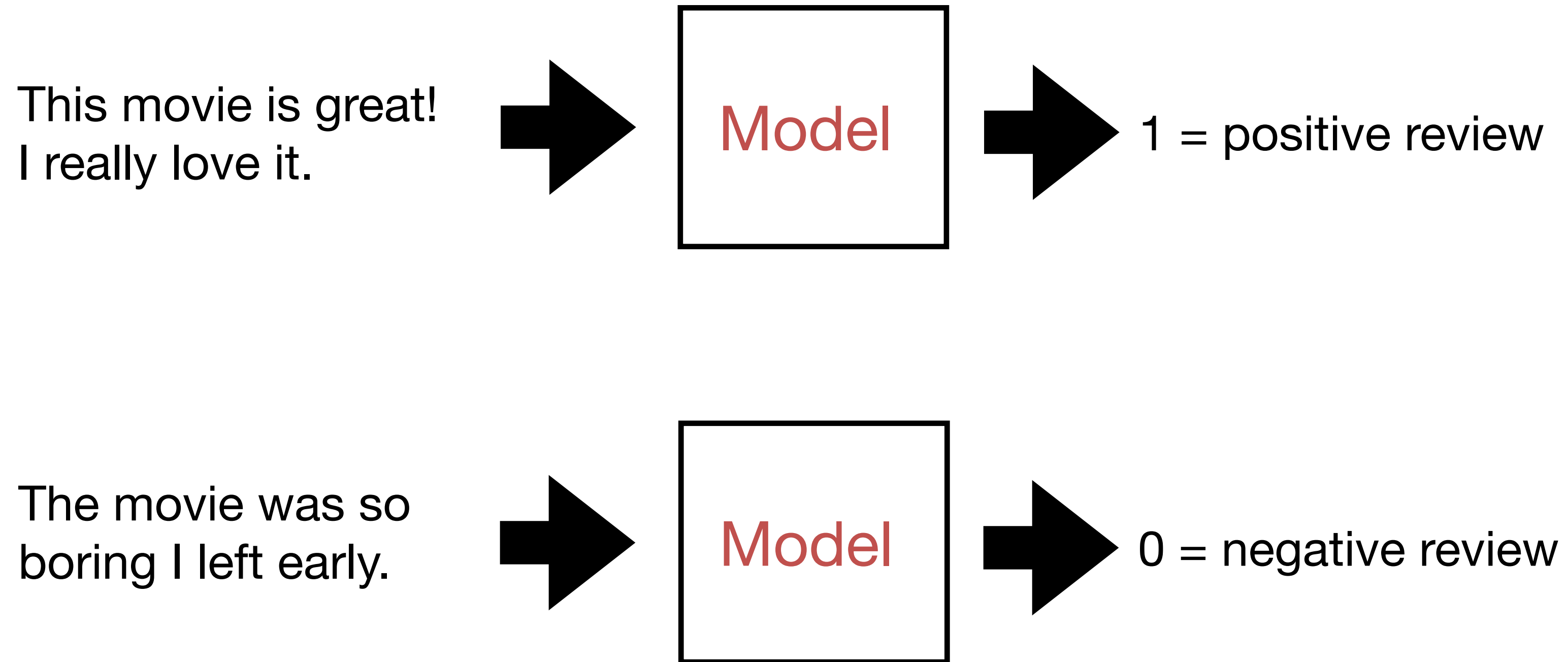
1. Binary classification of movie reviews.

2. Multi-class classification of newswires.

3. Regression for predicting house prices.



Example of **Binary Classification**: Classifying Movie Reviews



Get dataset

IMDB dataset: 50,000 highly polarized reviews on movies.

Balanced dataset:

50% positive reviews, 50% negative reviews.

25,000 reviews for training, and 25,000 reviews for testing.

Samples of IMDB dataset:

One of the other reviewers has mentioned that after watching just 1 Oz episode you'll be hooked. The...

Label: positive

A wonderful little production.The filming technique is very unassuming- very old-time-B...

Label: positive

Phil the Alien is one of those quirky films where the humour is based around the oddness of everything...

Label: negative

Get dataset

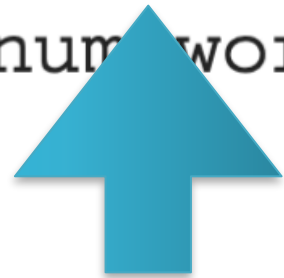
```
from tensorflow.keras.datasets import imdb  
(train_data, train_labels), (test_data, test_labels) = imdb.load_data(  
    num_words=10000)
```



Only keep the top 10,000 most frequent words in the training data.

Get dataset

```
from tensorflow.keras.datasets import imdb  
(train_data, train_labels), (test_data, test_labels) = imdb.load_data(  
    num_words=10000)
```



List of reviews



List of reviews

Get dataset

```
from tensorflow.keras.datasets import imdb
(train_data, train_labels), (test_data, test_labels) = imdb.load_data(
    num_words=10000)
```

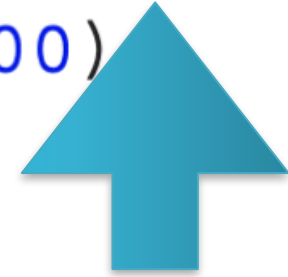
```
>>> train_data[0]
[1, 14, 22, 16, ... 178, 32]
```

Each review: a list of word indices.

Each word has been mapped to an integer.

Get dataset

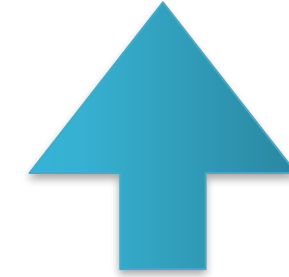
```
from tensorflow.keras.datasets import imdb  
(train_data, train_labels), (test_data, test_labels) = imdb.load_data(  
    num_words=10000)
```



List of 0s and 1s

0: negative review

1: positive review



List of 0s and 1s

Multi-hot Encode each review

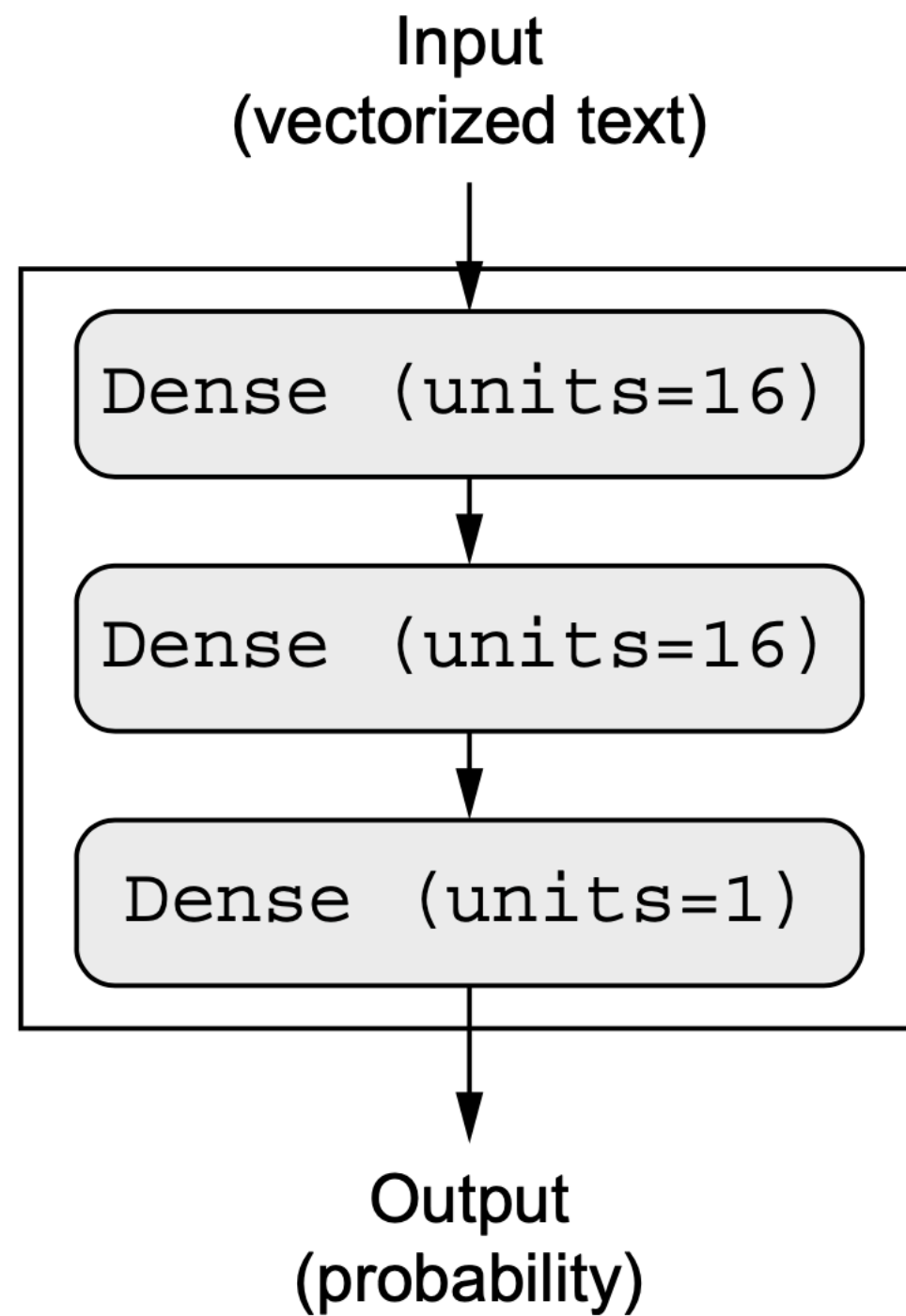
Turn each review into a 0-1 vector of length 10,000.

A bit in the vector is 1 if and only if its corresponding word is in the review.

Note: reviews can have different lengths, but the vectors are all of the same length.

Turn reviews and labels into real numbers (instead of integers).

Build the model



```
from tensorflow import keras
from tensorflow.keras import layers

model = keras.Sequential([
    layers.Dense(16, activation="relu"),
    layers.Dense(16, activation="relu"),
    layers.Dense(1, activation="sigmoid")
])
```


ReLU

$$\text{relu}(x) = \max(x, 0)$$

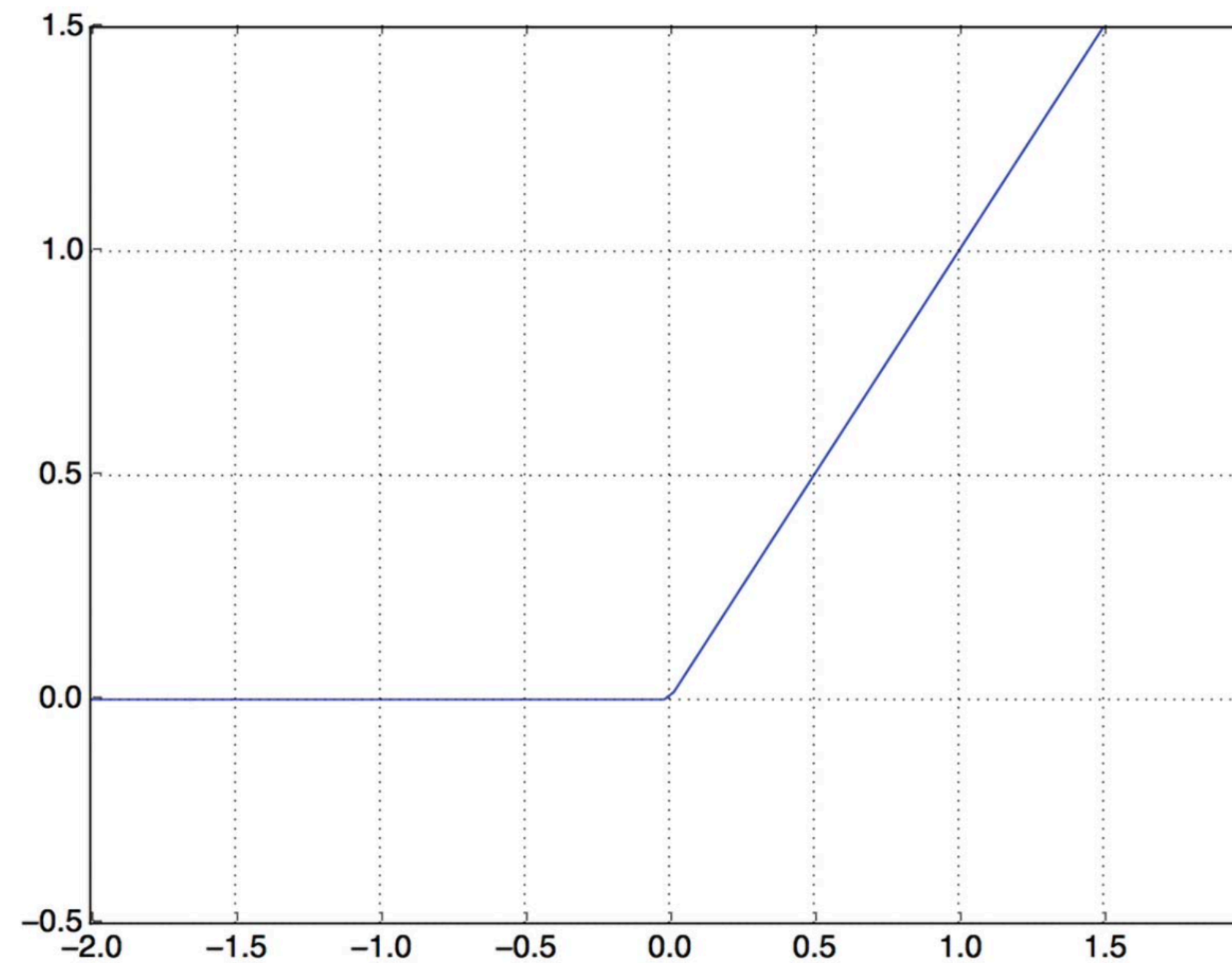


Figure 4.2 The rectified linear unit function

Sigmoid

$$\text{sigmoid}(x) = \frac{1}{1 + e^{-x}}$$

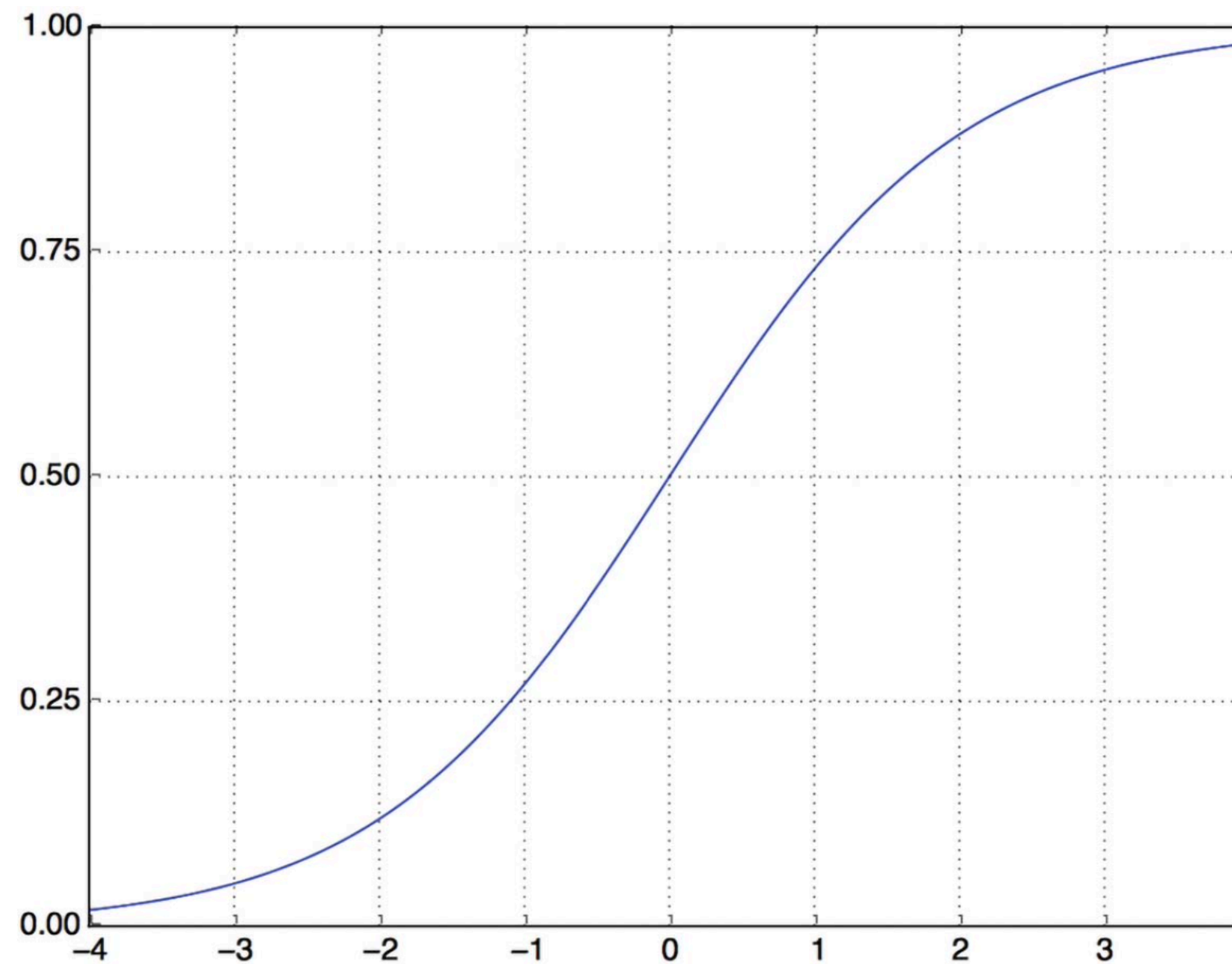


Figure 4.3 The sigmoid function

Compile the model

```
model.compile(optimizer="rmsprop",  
              loss="binary_crossentropy",  
              metrics=["accuracy"])
```

$$\text{binary crossentropy} = -\frac{1}{N} \sum_{i=1}^N L_i \log y_i + (1 - L_i) \log(1 - y_i)$$

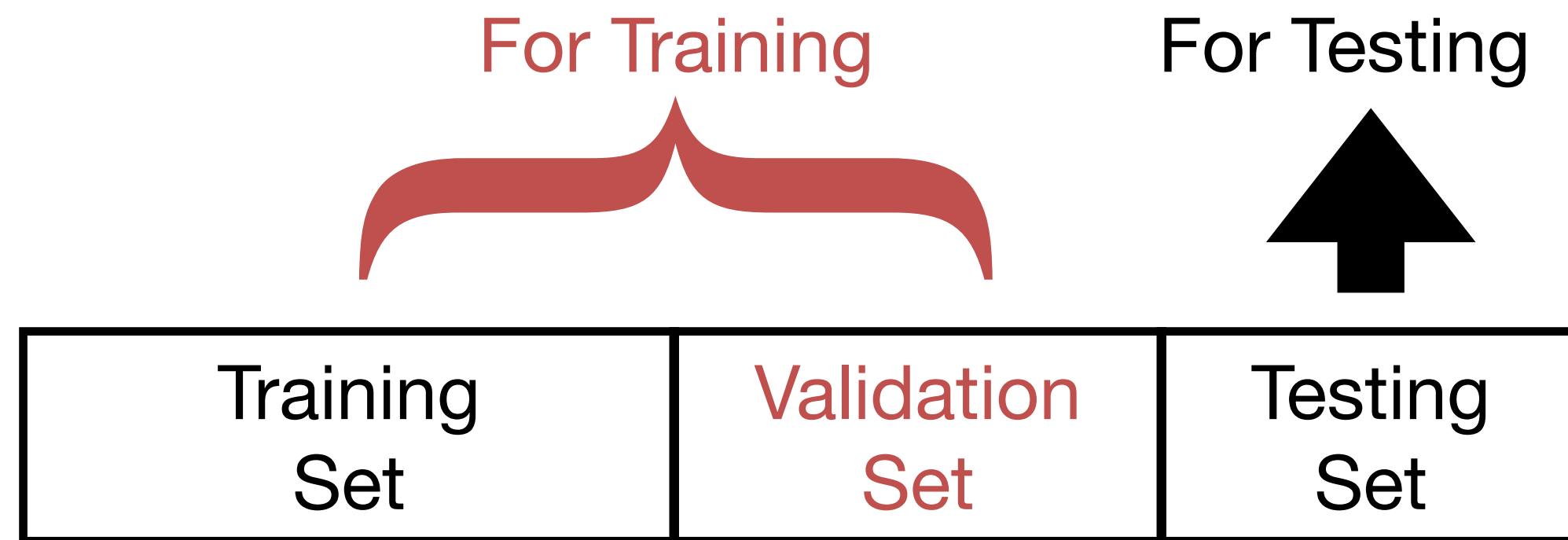
N : the number of samples in a mini-batch

$L_i \in \{0,1\}$: label of the i -th sample

$y_i \in (0,1)$: model's output for the i -th sample

The binary crossentropy is minimized when $y_i = L_i$ for all i

Validation Set



Listing 4.6 Setting aside a validation set

```
x_val = x_train[:10000]  
partial_x_train = x_train[10000:]  
y_val = y_train[:10000]  
partial_y_train = y_train[10000:]
```

Train the model with Validation Set

Listing 4.7 Training your model

```
history = model.fit(partial_x_train,  
                    partial_y_train,  
                    epochs=20,  
                    batch_size=512,  
                    validation_data=(x_val, y_val))
```

```
>>> history_dict = history.history  
>>> history_dict.keys()  
[u"accuracy", u"loss", u"val_accuracy", u"val_loss"]
```

Plot loss and accuracy

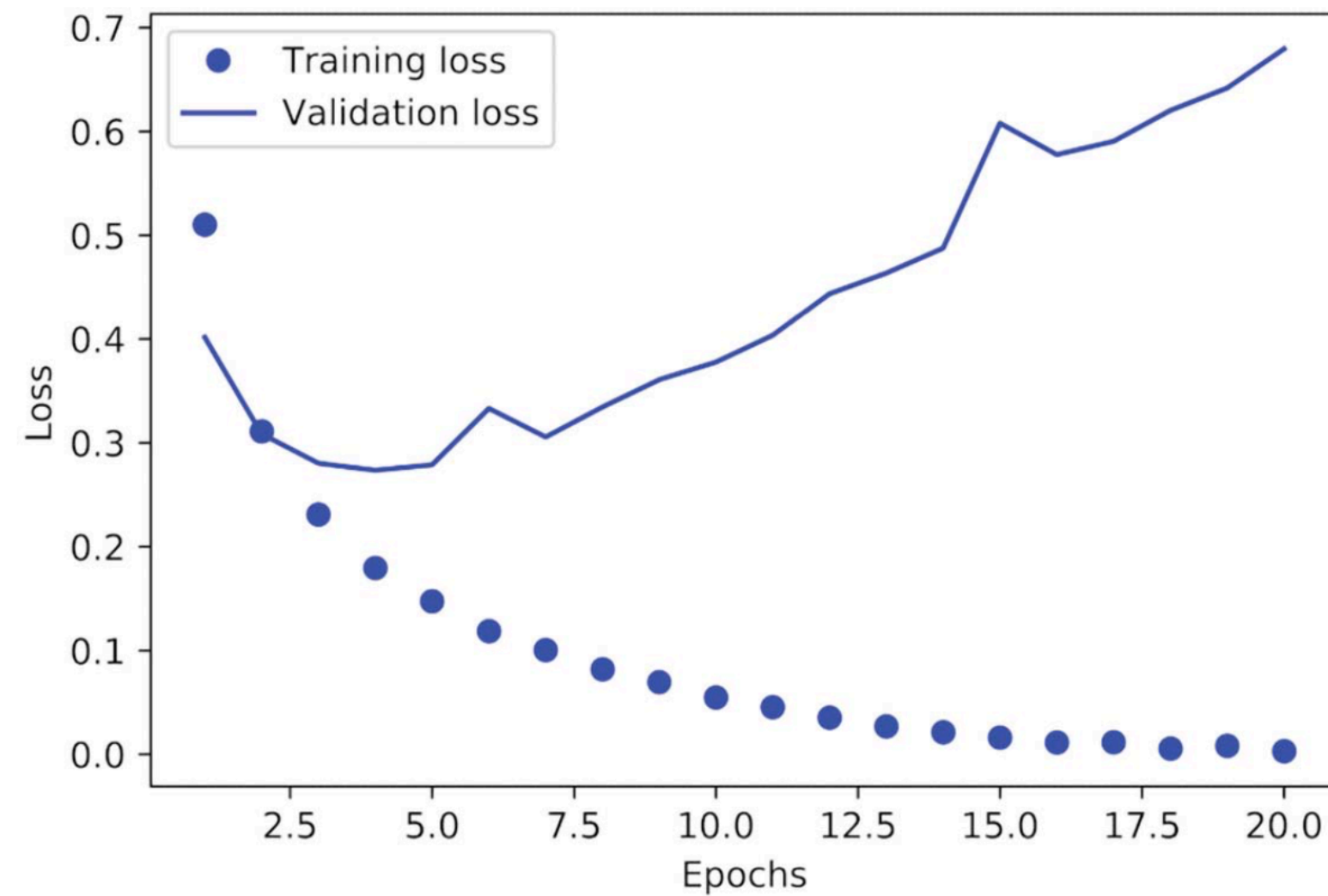


Figure 4.4 Training and validation loss

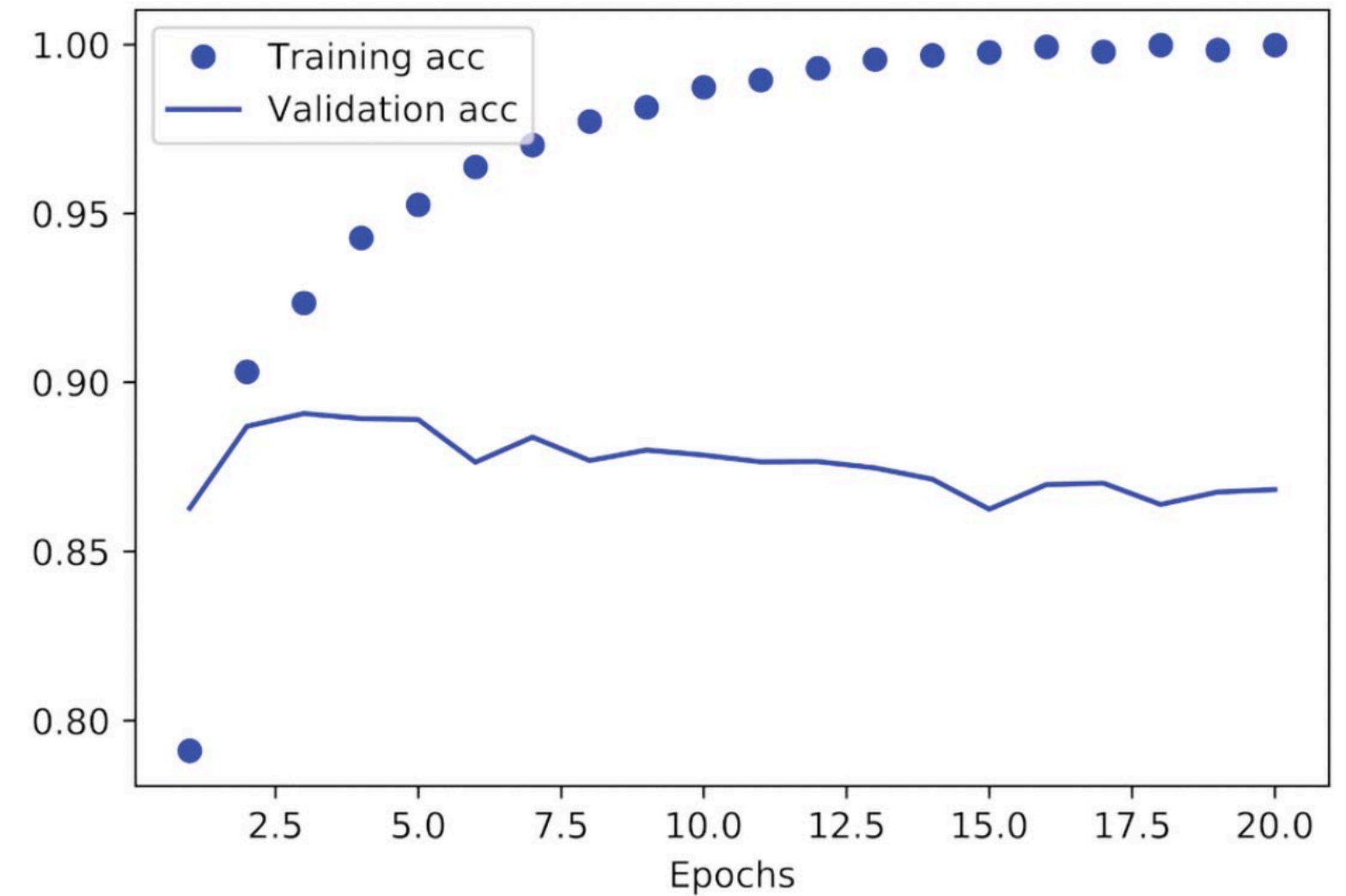
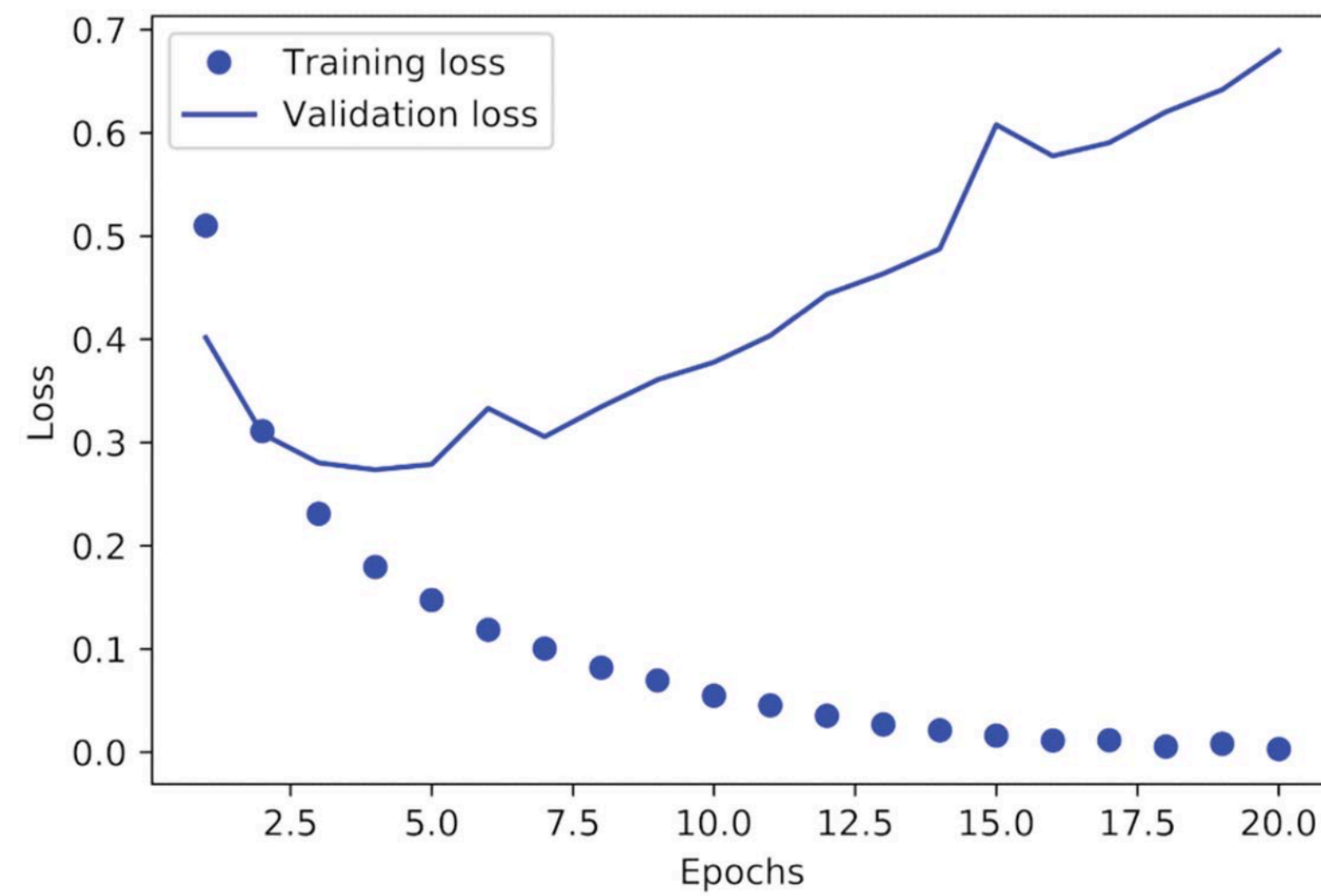


Figure 4.5 Training and validation accuracy

Overfit

Overfit

Underfit



Re-train the model for 4 epochs.

Use the trained model to generate prediction on new data

```
>>> model.predict(x_test)
array([[ 0.98006207]
       [ 0.99758697]
       [ 0.99975556]
       ...,
       [ 0.82167041]
       [ 0.02885115]
       [ 0.65371346]], dtype=float32)
```

Quiz questions:

1. What is the importance of having a balanced dataset?
2. What is the use of multi-hot encoding?
3. How to tell if overfitting happened during training?

Roadmap of this lecture:

1. Binary classification of movie reviews.

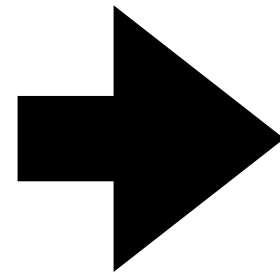
2. Multi-class classification of newswires.

3. Regression for predicting house prices.

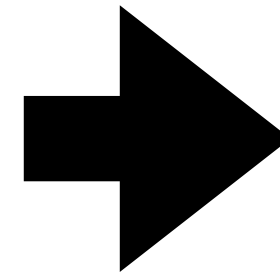


Example of **Multi-Class Classification**: Classifying Newswires

The copper price
just went up again
for the third straight
day. The stock market
...



Model



Topic 0: cocoa
Topic 1: grain
Topic 2: veg-oil
Topic 3: earn
Topic 4: acq
Topic 5: wheat
Topic 6: copper
Topic 7: housing
Topic 8: money-supply
Topic 9: coffee
Topic 10: sugar
Topic 11: trade
Topic 12: reserves
Topic 13: ship
Topic 14: cotton
Topic 15: carcass
...
Topic 45: lead

Get Dataset

Listing 4.11 Loading the Reuters dataset

```
from tensorflow.keras.datasets import reuters
(train_data, train_labels), (test_data, test_labels) = reuters.load_data(
    num_words=10000)
```

Reuters Dataset:

- Short newswires

- Each newswire has one topic

- 46 topics in total

Get Dataset

You have 8,982 training examples and 2,246 test examples:

```
>>> len(train_data)
8982
>>> len(test_data)
2246
```

As with the IMDB reviews, each example is a list of integers (word indices):

```
>>> train_data[10]
[1, 245, 273, 207, 156, 53, 74, 160, 26, 14, 46, 296, 26, 39, 74, 2979,
3554, 14, 46, 4689, 4329, 86, 61, 3499, 4795, 14, 61, 451, 4329, 17, 12]
```

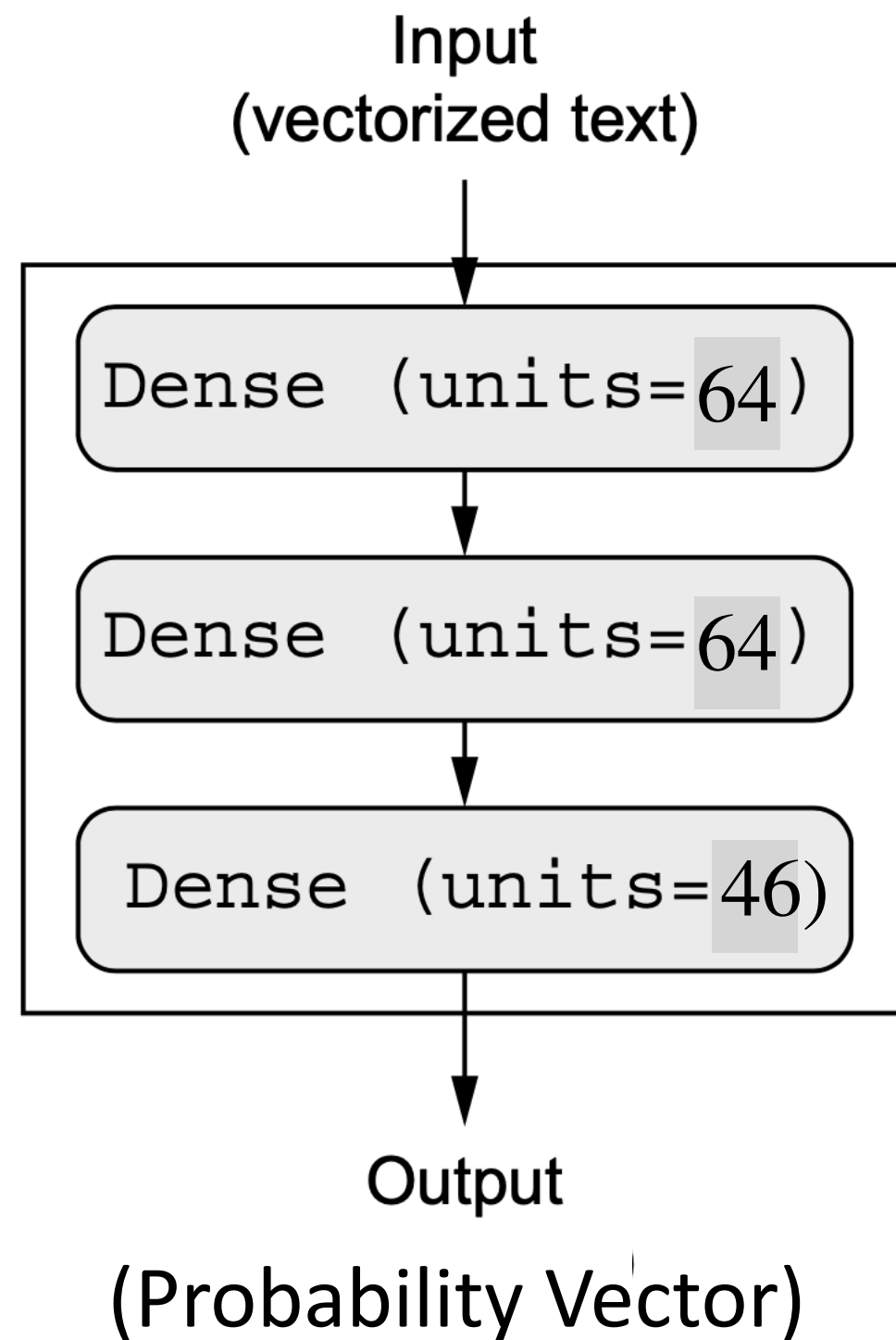
The label associated with an example is an integer between 0 and 45—a topic index:

```
>>> train_labels[10]
3
```

Vectorize input data using **Multi-Hot Encoding**

Vectorize labels using **One-Hot Encoding**

Build the model

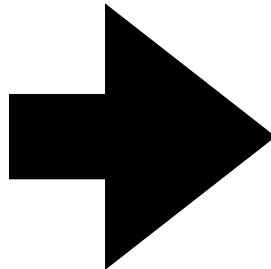


Listing 4.15 Model definition

```
model = keras.Sequential([
    layers.Dense(64, activation="relu"),
    layers.Dense(64, activation="relu"),
    layers.Dense(46, activation="softmax")
])
```


Softmax

x_1
 x_2
•
•
•
 x_n



$y_1 = \frac{e^{x_1}}{\sum_{i=1}^n e^{x_i}}$
 $y_2 = \frac{e^{x_2}}{\sum_{i=1}^n e^{x_i}}$
•
•
•
 $y_n = \frac{e^{x_n}}{\sum_{i=1}^n e^{x_i}}$

Compile the model

Listing 4.16 Compiling the model

```
model.compile(optimizer="rmsprop",  
              loss="categorical_crossentropy",  
              metrics=["accuracy"])
```

categorical crossentropy =

$$-\frac{1}{N} \sum_{i=1}^N (L_{i,1} \log y_{i,1} + L_{i,2} \log y_{i,2} + \cdots + L_{i,n} \log y_{i,n})$$

The categorical crossentropy is minimized when $y_{i,j} = L_{i,j}$ for all i, j

Check performance of training

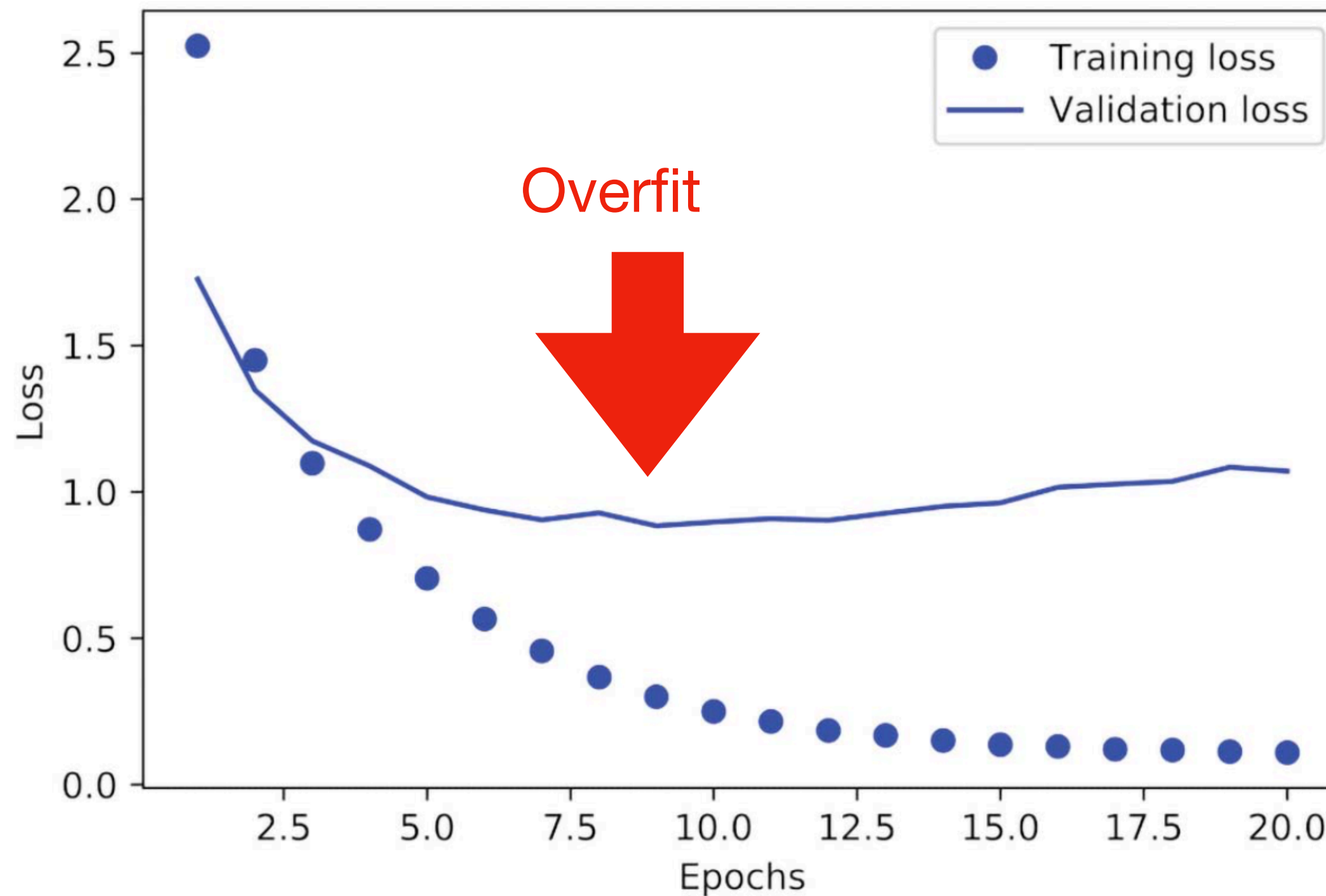


Figure 4.6 Training and validation loss

Re-train the model for 9 epochs.

Quiz questions:

1. Why do we use cross-entropy as the loss function?
2. What is the use of the “softmax” activation function?

Roadmap of this lecture:

1. Binary classification of movie reviews.
2. Multi-class classification of newswires.
3. Regression for predicting house prices.



Example of **Regression**: Predict house price

- 1) CRIM: per capita crime rate by town
- 2) ZN: proportion of residential land zoned for lots over 25,000 sq.ft.
- 3) INDUS: proportion of non-retail business acres per town.
- 4) CHAS: Charles River dummy variable (1 if tract bounds river; 0 otherwise)
- 5) NOX: nitric oxides concentration (parts per 10 million)
- 6) RM: average number of rooms per dwelling
- 7) AGE: proportion of owner-occupied units built prior to 1940
- 8) DIS: weighted distances to five Boston employment centres
- 9) RAD: index of accessibility to radial highways
- 10) TAX: full-value property-tax rate per \$10,000
- 11) PTRATIO: pupil-teacher ratio by town
- 12) LSTAT: % lower status of the population
- 13) MEDV: Median value of owner-occupied homes in \$1000's



Get Dataset

Boston Housing Price Dataset: only 506 samples, where each sample has 13 numerical features.

Listing 4.23 Loading the Boston housing dataset

```
from tensorflow.keras.datasets import boston_housing
(train_data, train_targets), (test_data, test_targets) = (
    boston_housing.load_data())
```

Let's look at the data:

```
>>> train_data.shape
(404, 13)
>>> test_data.shape
(102, 13)
```

The targets are the median values of owner-occupied homes, in thousands of dollars:

```
>>> train_targets
[ 15.2,  42.3,  50. ...  19.4,  19.4,  29.1]
```

Prepare the data

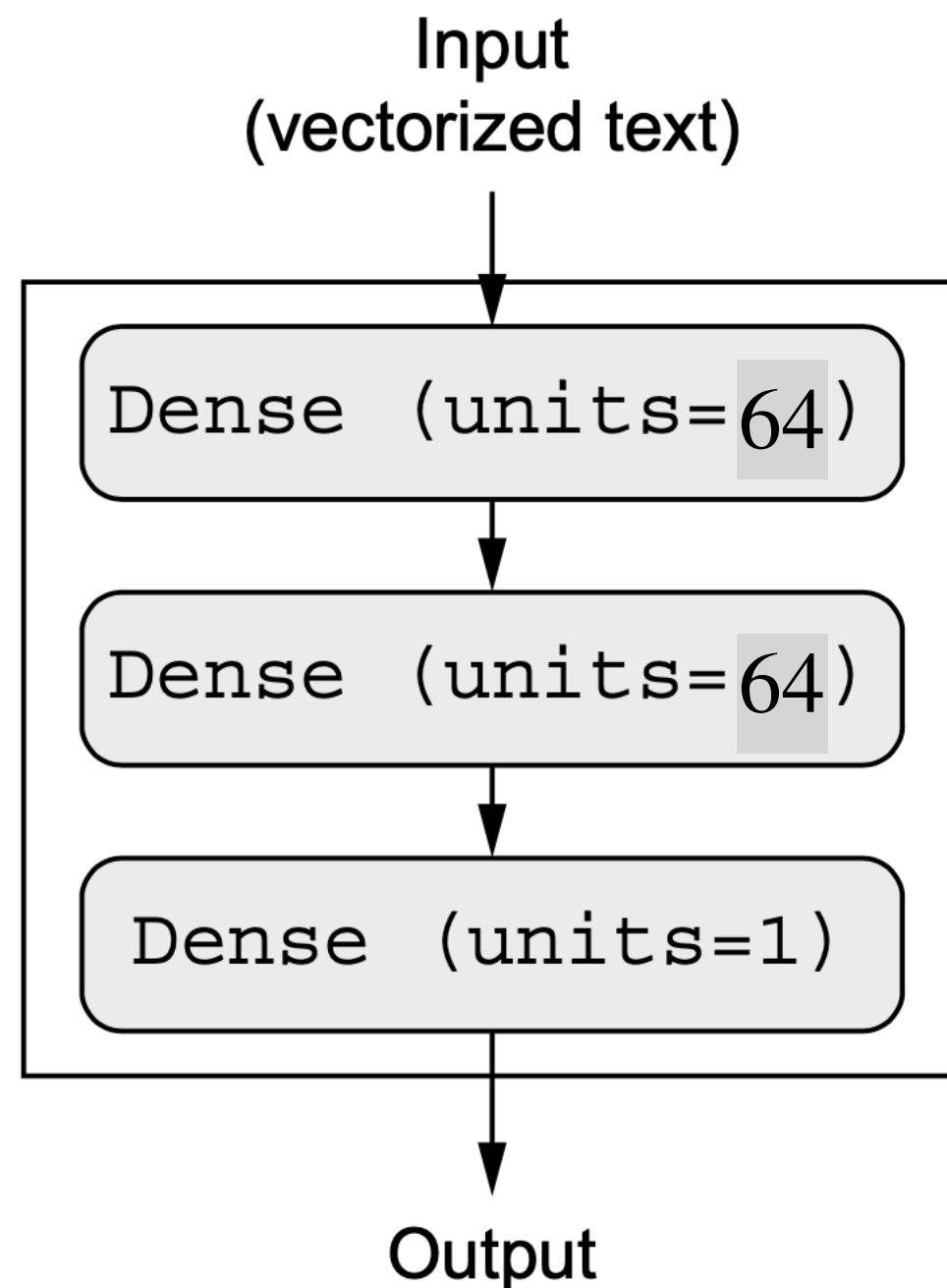
Normalize the data so that each feature has mean 0 and standard deviation 1.

Listing 4.24 Normalizing the data

```
mean = train_data.mean(axis=0)
train_data -= mean

std = train_data.std(axis=0)
train_data /= std
test_data -= mean
test_data /= std
```


Build the model, and compile the model



Listing 4.25 Model definition

```
def build_model():  
    model = keras.Sequential([  
        layers.Dense(64, activation="relu"),  
        layers.Dense(64, activation="relu"),  
        layers.Dense(1)  
    ])  
    model.compile(optimizer="rmsprop", loss="mse", metrics=["mae"])  
    return model
```

Because we need to instantiate the same model multiple times, we use a function to construct it.

MSE: mean square error

$$\text{MSE} = \frac{1}{N} \sum_{i=1}^N (L_i - y_i)^2$$

MAE: mean absolute error

$$\text{MAE} = \frac{1}{N} \sum_{i=1}^N |L_i - y_i|$$

K-fold validation for small dataset

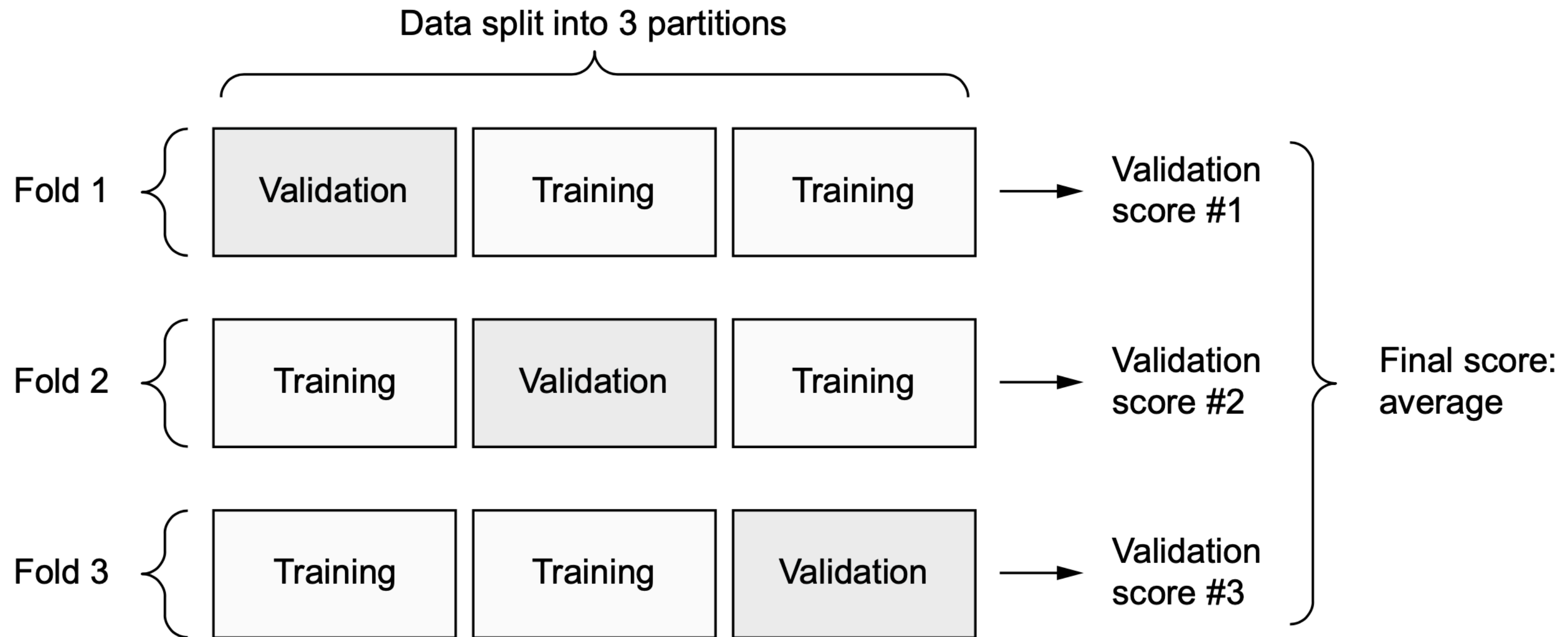


Figure 4.8 K-fold cross-validation with K=3

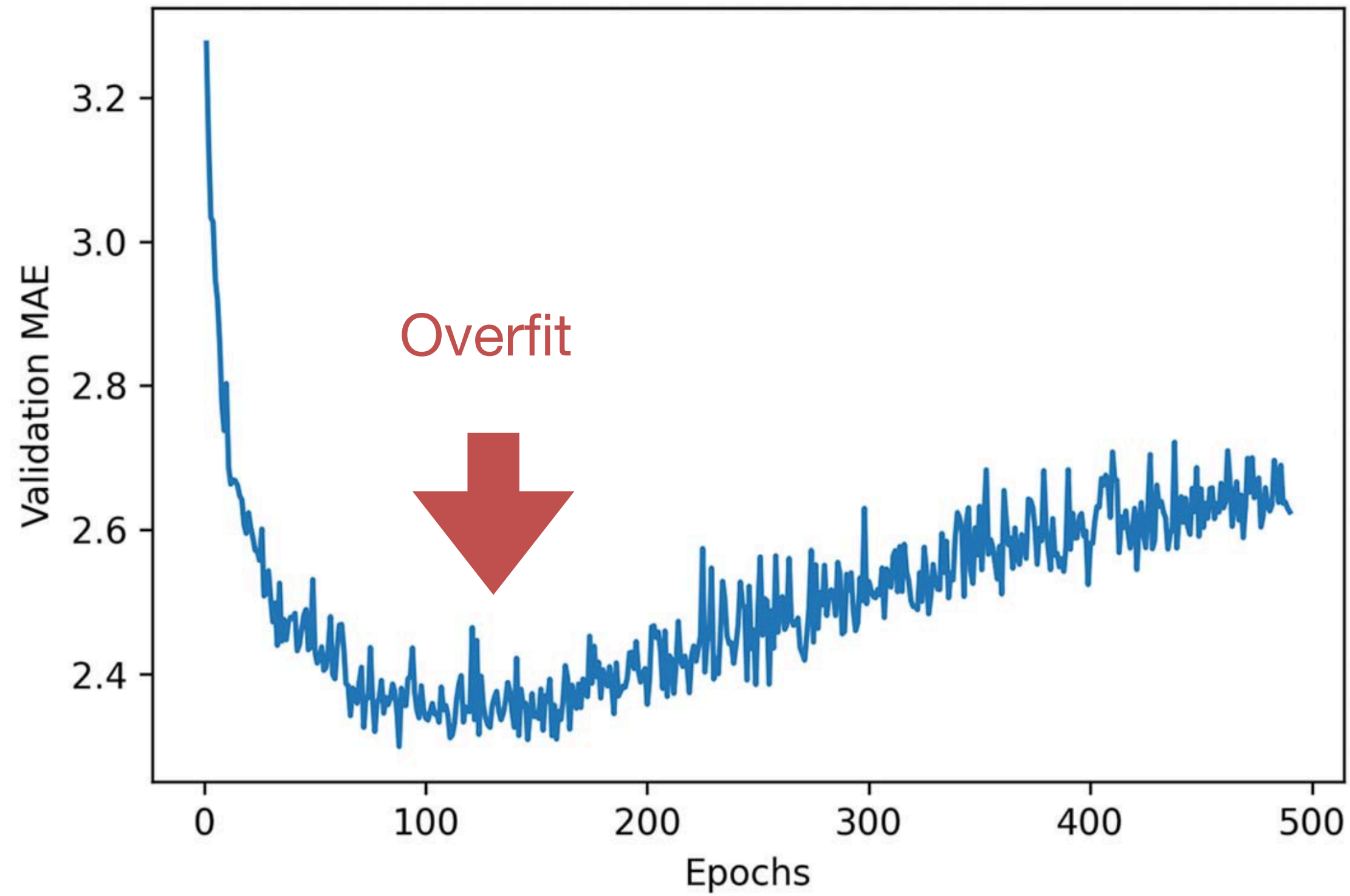


Figure 4.10 Validation MAE by epoch, excluding the first 10 data points

Re-train the model for 130 epochs.

Quiz questions:

1. What is the use of K-fold validation?
2. What are the differences between regression and classification?